



KOMMURI PRATAP REDDY INSTITUTE OF TECHNOLOGY
(UGC-Autonomous Institution)

Ghanpur(V), Ghatkesar(M), Medchal(D), Hyderabad-500088

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (AI&ML)

MAJOR PROJECT STAGE-1, REVIEW-2 REPORT

ROLE-BASED MULTI-AGENT LLMs for Transparent Decision-Making

*A transparent decision-support system using custom Flask/Python orchestration, the Gemini API,
Mock AI Mode, and four role-based agents: Advocate, Skeptic, Compliance Checker, and
Adjudicator*

Under the Guidance of

Dr. Pulime Satyanarayana

Professor

Department of CSE (AI&ML)

Presented by

Name	Roll Number
N. Vishnu Kumar	23RA1A66D4
G. Vinay	23RA1A66D1
M. Yeshwanth	23RA1A66E0
K. Shiva Sai	23RA1A66A5

1. System Architecture

The input case enters the multi-agent debate layer, where four role-based agents work over two rounds. A bias-probing module runs in parallel. The Adjudicator's decision, with its reason, explanation, and confidence score, is written to a JSON audit log and stored in SQLite.

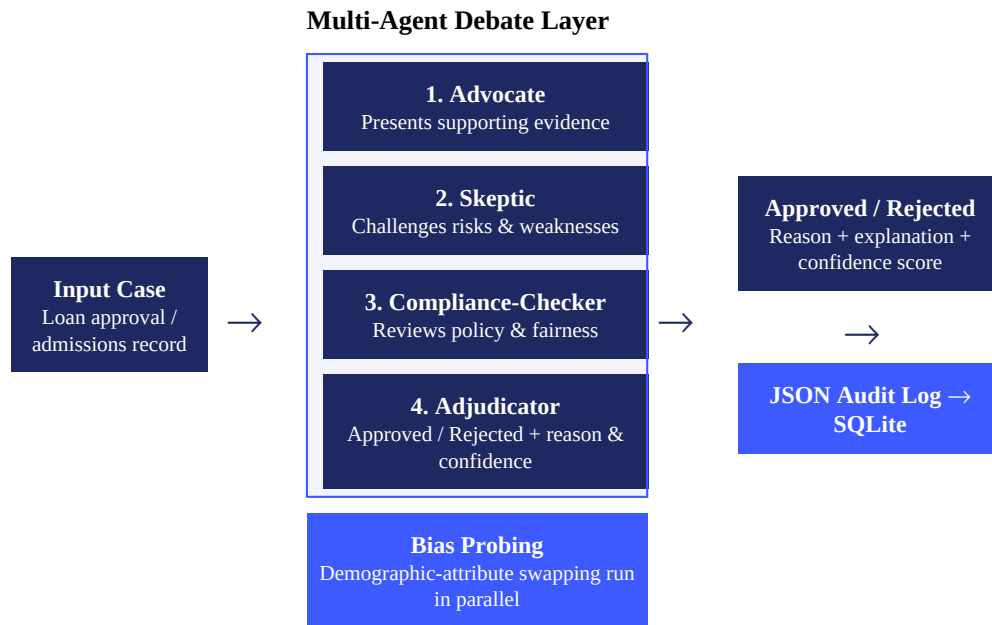


Figure 1: Architecture and data flow

Each component of the architecture is explained below.

1.1. Input Case

A loan application or an academic-admission record is submitted to the system through the Flask API layer, where form validation checks the input before processing. Only synthetic records are used; no real student data is involved.

1.2. Advocate Agent

Argues in favor of the applicant or case and presents the supporting evidence from the record, giving the debate its positive case.

1.3. Skeptic Agent

Challenges risks, weaknesses, and unsupported reasoning in the case and in the Advocate's arguments, so that the decision is tested before it is finalized.

1.4. Compliance Checker Agent

Reviews the case and the debate for policy and fairness concerns, and records its compliance findings. It reviews in Round 1 and performs a final review in Round 2.

1.5. Adjudicator Agent

Reads the full debate and produces the final binary decision, Approved or Rejected, together with a reason, an explanation, and a confidence score.

1.6. Bias Probing Module

Runs in parallel with the debate. It swaps demographic attributes across otherwise identical profiles and compares the resulting decisions to detect potential inconsistency and bias-related outcomes. It reports potential issues and does not guarantee fairness.

1.7. Language Model Backend

The Gemini API powers the agents. Mock AI Mode replaces it when no API connection is available, so the pipeline can be tested without calling the API.

1.8. Audit Log and Storage

Agent outputs, debate history, compliance findings, confidence scores, and the final decision are recorded as structured JSON audit logs, and cases and results are stored in SQLite.

1.9. Output and Dashboard

The decision, reason, explanation, and confidence score are returned to the user. The dashboard shows approval/rejection counts, Decision Consistency, Bias Alert, Fairness Percentage, and Average Confidence.

2. Methodology

The system follows a debate-driven decision workflow. Each stage is recorded in the audit log, giving a full trail from applicant input to final decision.



Figure 2: Debate-driven decision workflow

The steps of the workflow are explained below.

Step 1. Applicant Input

The applicant's loan or admission details are entered and validated. This is the case that the committee will debate.

Step 2. Loan or Admission Evaluation

The case is passed to the loan or the academic-admission evaluation module, which prepares it for the debate engine.

Step 3. Debate Round 1

The Advocate argues for the case, the Skeptic challenges its risks and weaknesses, and the Compliance Checker reviews policy and fairness concerns.

Step 4. Debate Round 2

The agents respond to the arguments made in Round 1, and the Compliance Checker performs a final review before the case goes to the Adjudicator.

Step 5. Adjudication

The Adjudicator synthesizes the two rounds and issues the final decision, Approved or Rejected, with a reason, an explanation, and a confidence score.

Step 6. Audit Logging and Storage

The audit record stores agent outputs, debate history, compliance findings, confidence scores, and final decision explanations as JSON, and the case and result are stored in SQLite.

Step 7. Bias Probing and Evaluation

Demographic attributes are swapped across otherwise identical profiles to check decision consistency. Evaluation uses 1,000 synthetic loan records and 1,000 synthetic academic-admission records, with approval/rejection counts, Decision Consistency, Bias Alert, Fairness Percentage, Average Confidence, and agent confidence scores as metrics.

3. References

- [1] Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2023). Improving Factuality and Reasoning in Language Models through Multiagent Debate. arXiv:2305.14325.
- [2] Wu, Q., Bansal, G., Zhang, J., et al. (2024). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations. First Conference on Language Modeling (COLM).
- [3] Benchmarking Large Language Models for Multi-Agent Systems: A Comparative Analysis of AutoGen, CrewAI, and TaskWeaver (2024). arXiv preprint.
- [4] Weidinger, L., Uesato, J., Rauh, M., et al. (2022). Taxonomy of Risks Posed by Language Models. ACM Conference on Fairness, Accountability, and Transparency (FAccT), 214–229.
- [5] Emergent Bias and Fairness in Multi-Agent Decision Systems (2025). arXiv:2512.16433.
- [6] Emergence of Biased Consensus in Multi-Agent LLM Debates (2026). arXiv:2608.02827.
- [7] Google (n.d.). Gemini API Documentation. ai.google.dev/gemini-api/docs
- [8] Pallets Projects (n.d.). Flask Documentation. flask.palletsprojects.com
- [9] SQLite Consortium (n.d.). SQLite Documentation. sqlite.org/docs.html